

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
er_blob_replication.py – Réplication du scaling ER-Blob
Blob Principle – Rudolphe Mirante – @Blob_Principle
```

DÉFI DE RÉPLICATION OUVERT

Ce script reproduit l'un des résultats mesurés [M] du Blob Principle : la loi de puissance $\tau_{\text{sym}} \propto N^{1,09}$ pour le temps de demi-vie de la symétrie miroir d'un pont d'Einstein-Rosen géométrique.

USAGE LOCAL :

```
pip install numpy scipy networkx
python3 er_blob_replication.py
```

USAGE COLAB (1-click) :

Ouvrir dans Google Colab, runtime CPU suffit.

PRÉDICTION TESTABLE

Pour $N = 1000$, $\kappa_0 = 8$, $\beta = 0.30$, $\tau = 0.50$, seuil $\Delta_{\text{sym}} = 0.40$:

τ_{sym} médiane (10 seeds) = 4510 ± 150 pas MCMC

Contrôle aléatoire (graphe Rips sans condition miroir) : $\tau_{\text{sym}} = 0$.

Si votre mesure diffère significativement (>300 pas sur 10 seeds), signalez-le : l'objectif est de casser le résultat, pas de le confirmer.

```
"""
```

```
import sys, time
import numpy as np
from scipy.spatial import distance_matrix
import networkx as nx
```

```
# — Paramètres —
N          = 1000          # Taille du graphe
```

```

KAPPA0      = 8          # Degré moyen cible
BETA        = 0.30       # Coefficient de dissipation dans S_sim
TAU         = 0.50       # Température effective
SYM_THRESH  = 0.40       # Seuil de brisure de symétrie
N_SEEDS     = 10         # Nombre de seeds indépendants
MAX_STEPS   = 10000      # Longueur maximale de chaîne MCMC

# ——— Structure de graphe sparse —————
class SparseBlob:
    """Graphe Blob avec adjacency-sets pour MCMC  $O(k)$  par étape."""

    def __init__(self, N, adj_matrix):
        self.N          = N
        self.N_half     = N // 2
        self.adj         = [set() for _ in range(N)]
        self.edges       = []
        self.edge_set    = set()
        self.deg         = np.zeros(N, dtype=int)
        self.n_tri       = 0

        for i in range(N):
            for j in range(i+1, N):
                if adj_matrix[i, j]:
                    self._add(i, j, update_tri=False)

        # Compte les triangles une fois
        self.n_tri = 0
        for u, v in self.edge_set:
            self.n_tri += len(self.adj[u] & self.adj[v])
        self.n_tri //= 3

    def _add(self, u, v, update_tri=True):
        if update_tri:
            self.n_tri += len(self.adj[u] & self.adj[v])
        self.adj[u].add(v); self.adj[v].add(u)
        key = (min(u,v), max(u,v))
        self.edges.append(key)
        self.edge_set.add(key)
        self.deg[u] += 1; self.deg[v] += 1

    def _remove(self, u, v):
        self.n_tri -= len((self.adj[u] & self.adj[v]) - {u, v})
        self.adj[u].discard(v); self.adj[v].discard(u)
        key = (min(u,v), max(u,v))
        self.edge_set.discard(key)
        for idx in range(len(self.edges)-1, -1, -1):
            if self.edges[idx] == key:

```

```

        self.edges[idx] = self.edges[-1]
        self.edges.pop()
        break
    self.deg[u] -= 1; self.deg[v] -= 1

def D(self):
    mu = float(self.deg.mean())
    if mu < 1e-9: return 0.0
    return float(self.deg.std()) / mu

def Ssim(self, beta=BETA):
    return np.log(1 + self.n_tri) - beta * self.D()

def sym_deviation(self):
    """|E_AA Δ mirror(E_BB)| / |E_AA ∪ mirror(E_BB)|"""
    edges_A = {(u, v) for u, v in self.edge_set
                if u < self.N_half and v < self.N_half}
    edges_B = {(u - self.N_half, v - self.N_half)
                for u, v in self.edge_set
                if u >= self.N_half and v >= self.N_half}
    union = edges_A | edges_B
    if not union: return 0.0
    sym_diff = edges_A ^ edges_B
    return len(sym_diff) / len(union)

def is_still_connected(self, u, v):
    """BFS depuis u pour atteindre v."""
    visited = {u}
    stack = list(self.adj[u])
    while stack:
        node = stack.pop()
        if node == v: return True
        if node not in visited:
            visited.add(node)
            stack.extend(self.adj[node] - visited)
    return False

def random_edge(self, rng):
    return self.edges[rng.integers(len(self.edges))]

def random_non_edge(self, rng, tries=200):
    for _ in range(tries):
        u = int(rng.integers(self.N))
        v = int(rng.integers(self.N))
        if u != v and v not in self.adj[u]:
            return min(u,v), max(u,v)
    return None

```

```

# — Construction configuration miroir —————
def build_mirror(N, kappa0=KAPPA0, seed=42):
    """Configuration initiale symétrique par rapport à x=0.5"""
    rng = np.random.default_rng(seed)
    N_half = N // 2
    pts_A = rng.uniform(0.0, 0.45, (N_half, 3))
    pts_B = 1.0 - pts_A
    pts = np.vstack([pts_A, pts_B])
    r = (3 * kappa0 / (4 * np.pi * N)) ** (1/3)
    dist = distance_matrix(pts, pts)
    adj = (dist < r).astype(np.int8)
    np.fill_diagonal(adj, 0)
    return adj

# — Un run MCMC jusqu'à brisure de symétrie —————
def run_until_break(N, kappa0=KAPPA0, beta=BETA, tau=TAU,
                    sym_thresh=SYM_THRESH, max_steps=MAX_STEPS, seed=0):
    rng = np.random.default_rng(seed)
    adj = build_mirror(N, kappa0, seed=seed)
    G = SparseBlob(N, adj)

    check_every = max(1, N // 50)

    for step in range(max_steps):
        if step % check_every == 0:
            sym = G.sym_deviation()
            if sym > sym_thresh:
                return step

        if not G.edges:
            break

        e_rm = G.random_edge(rng)
        ne = G.random_non_edge(rng)
        if ne is None: continue
        u_rm, v_rm = e_rm
        u_add, v_add = ne

        dt_rm = len((G.adj[u_rm] & G.adj[v_rm]) - {u_rm, v_rm})
        G._remove(u_rm, v_rm)
        if not G.is_still_connected(u_rm, v_rm):
            G._add(u_rm, v_rm)
            continue
        dt_add = len(G.adj[u_add] & G.adj[v_add])
        G._add(u_add, v_add)

    delta_n = dt_add - dt_rm

```

```

    old_S    = np.log(1 + G.n_tri - delta_n) - beta * G.D()
    new_S    = G.Ssim(beta)
    delta_S = N * (new_S - old_S) / tau

    if delta_S < 0 and rng.random() >= np.exp(np.clip(delta_S, -500, 0)):
        G._remove(u_add, v_add)
        G._add(u_rm, v_rm)

    return max_steps

# — Main —————
def main():
    print("=" * 65)
    print("ER-Blob – Réplication ouverte (Blob Principle)")
    print("Rudolphe Mirante – @Blob_Principle")
    print("=" * 65)
    print(f"Paramètres : N={N},  $\kappa_0$ ={{KAPPA0}},  $\beta$ ={{BETA}},  $\tau$ ={{TAU}}")
    print(f"Seuil de brisure  $\Delta_{\text{sym}}$  > {{SYM_THRESH}}, max_steps={{MAX_STEPS}}")
    print(f"Seeds : {{N_SEEDS}} indépendants\n")

    print(f"Prédiction [M] attendue :  $\tau_{\text{sym}}$  médiane = 4510  $\pm$  150 pas")
    print("-" * 65)

    taus = []
    t_start = time.time()

    for seed in range(N_SEEDS):
        t0 = time.time()
        tau = run_until_break(N, seed=seed * 1000 + N)
        elapsed = time.time() - t0
        taus.append(tau)
        marker = " (CENSURE)" if tau >= MAX_STEPS else ""
        print(f"  seed {{seed:2d}}  $\rightarrow$   $\tau_{\text{sym}}$  = {{tau:5d}} pas  ({{elapsed:5.1f}}s){{marker}}
              flush=True)

    total = time.time() - t_start
    taus = np.array(taus)
    tau_med = float(np.median(taus))
    tau_mean = float(np.mean(taus))
    tau_std = float(np.std(taus))
    tau_min = int(np.min(taus))
    tau_max = int(np.max(taus))
    n_censored = int(np.sum(taus >= MAX_STEPS))

    print("-" * 65)
    print(f"RÉSULTATS ({{total:.0f}}s total)")
    print(f"  Médiane      : {{tau_med:.0f}} pas")

```

```

print(f"  Moyenne      : {tau_mean:.0f} ± {tau_std:.0f} pas")
print(f"  Min / Max    : {tau_min} / {tau_max}")
print(f"  Censurés      : {n_censored}/{N_SEEDS}")
print()
print("=" * 65)
expected = 4510
tolerance = 300
deviation = abs(tau_med - expected)
if deviation < tolerance:
    print(f"✅ RÉSULTAT COHÉRENT avec la prédiction [M]")
    print(f"  Médiane = {tau_med:.0f} pas (attendu 4510 ± {tolerance})")
else:
    print(f"⚠️ ÉCART AVEC LA PRÉDICTION")
    print(f"  Médiane = {tau_med:.0f} pas (attendu 4510 ± {tolerance})")
    print(f"  Écart de {deviation:.0f} pas – à signaler sur @Blob_Principe")
print("=" * 65)
print()
print("Contrôle aléatoire (graphe Rips sans condition miroir) : τ_sym = 0")
print("La symétrie miroir n'existe pas sans la condition initiale partagée.")
print()
print("Pour signaler une anomalie ou une erreur dans le script :")
print("  → @Blob_Principe sur X")
print("  → issue sur le dépôt GitHub (à venir)")

if __name__ == "__main__":
    main()

```